Project - High Level Design

On

RealEstate IQ bot

Course Name:

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	Samruddhi Muley	EN22CS304056
2	Payal Singh	EN22CS301701
3	Aarti Bamanke	EN22EE301001
4	Akshat Joshi	EN22CS304006
5	Punit Pawar	EN22CS303038
6	Shubhi Dubey	EN22CS304061

Group Name: 01D11

Project Number: DO-27

Industry Mentor Name:

University Mentor Name: Dr. Hemlata Patel

Academic Year: 2026

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. Data Design.
   3.1. Data Model
   3.2. Data Access Mechanism
   3.3. Data Retention Policies
   3.4. Data Migration

4. Interfaces

5. State and Session Management

6. Caching

7. Non-Functional Requirements
   7.1. Security Aspects
   7.2. Performance Aspects

8. References

1. Introduction

The **RealEstate IQ Bot** is an AI-powered quiz-based web application designed to help users learn real estate concepts interactively. The system leverages modern technologies such as Python, Streamlit, and Groq API.

1.1 Scope of the Document

This document defines:

- High-level architecture of the system
- Infrastructure and deployment design
- Data handling and flow
- API structure
- Security and performance considerations

This document **does not include low-level implementation details**.

1.2 Intended Audience

This document is intended for:

- DevOps Engineers
- Cloud Architects
- Backend Developers
- Project Evaluators
- Technical Reviewers

1.3 System Overview

The system follows a **modular and scalable architecture**, consisting of:

- **Frontend:** Streamlit-based UI
- **Backend:** Python application logic
- **AI Layer:** Groq API
- **Data Layer:** Question bank (static dataset)
- **CI/CD:** GitHub Actions (or Jenkins)

The system is designed to support:

- Automated deployment
- Scalability
- Secure API usage
- Interactive learning

---

2. System Design

2.1 Application Design

The system follows a **layered architecture**:

1. Presentation Layer

- Built using Streamlit
- Displays quiz, results, and dashboard

2. Application Layer

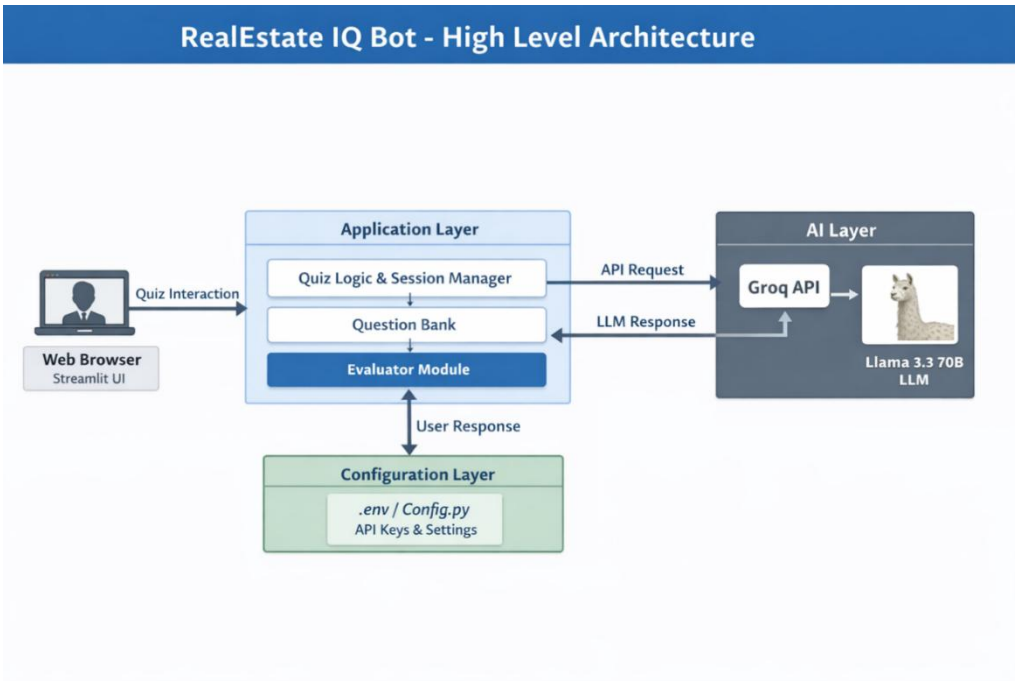
- Python backend logic
- Handles quiz flow and evaluation

3. AI Layer

- Integrates Google Gemini API
- Evaluates answers and provides explanations

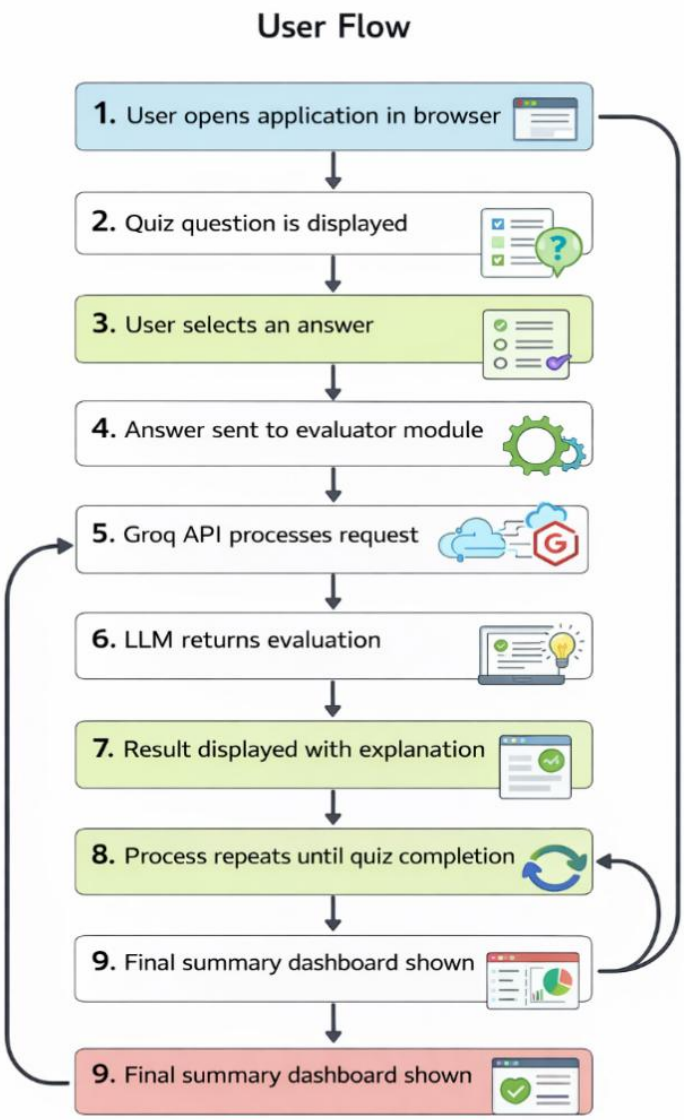
4. Data Layer

- Stores questions in question_bank.py
- Static structured dataset

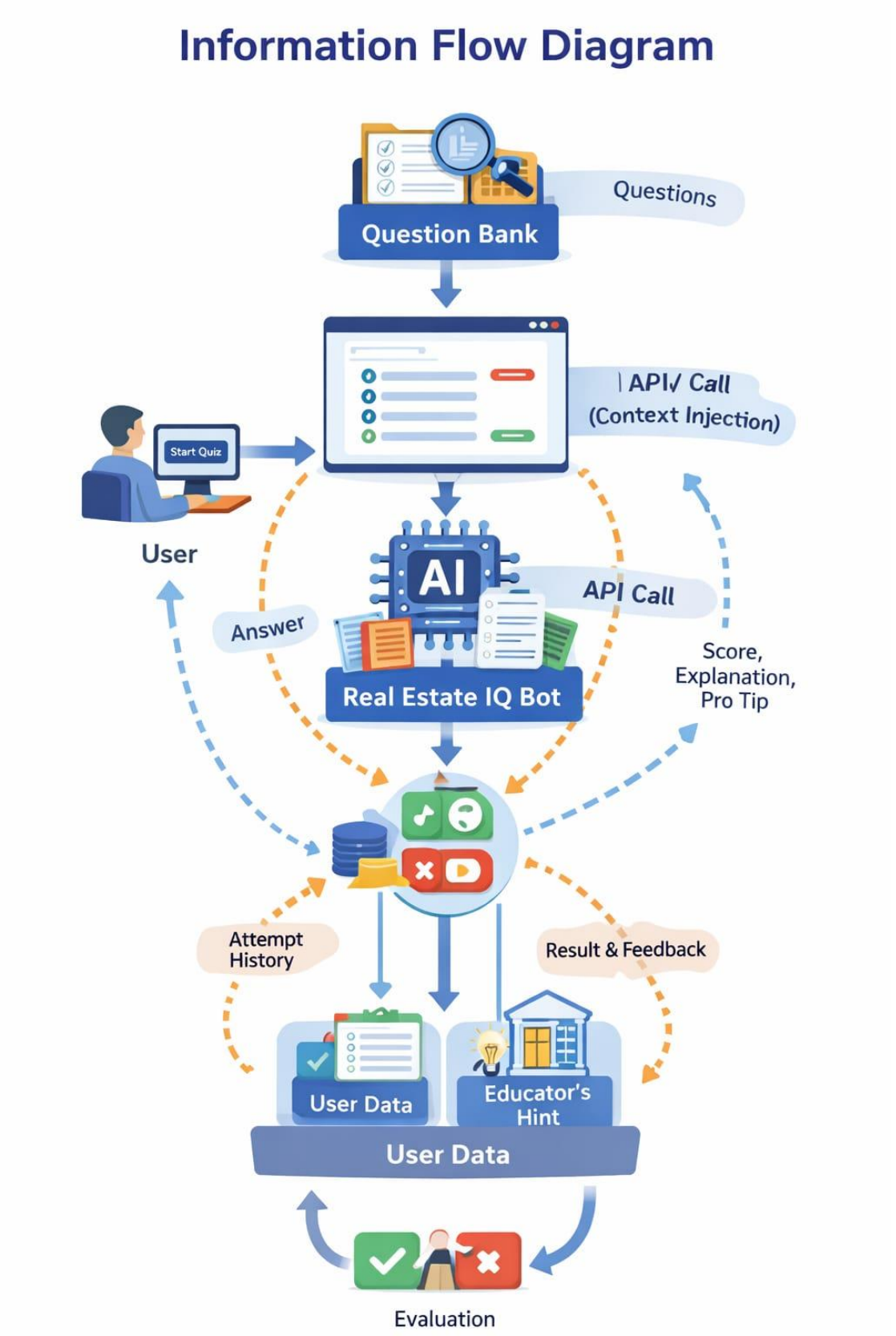


2.2 Process Flow

User Flow


User Flow
1. User opens application in browser
2. Quiz question is displayed
3. User selects an answer
4. Answer sent to evaluator module
5. Groq API processes request
6. LLM returns evaluation
7. Result displayed with explanation
8. Process repeats until quiz completion
9. Final summary dashboard shown
9. Final summary dashboard shown

2.3 Information Flow



2.4 Components Design

Core Components

- **Frontend (Streamlit)**
  - Displays UI and captures input
- **Backend (Python)**
  - Controls quiz logic
- **Evaluator Module**
  - Calls Groq API
- **Question Bank**
  - Stores quiz data

2.5 Key Design Considerations

- Modular architecture for scalability
- Secure API key management using .env
- Lightweight UI using Streamlit
- Error handling for API failures
- CI/CD for automated deployment
- Containerization for portability

2.6 API Catalogue

Internal API

Function	Description
Evaluate Answer	Sends response to Groq API
Get Question	Fetches question from dataset

External API

Groq API

- Evaluates answers

- Generates explanations
- Provides improvement tips

---

3. Data Design

The RealEstate IQ Bot uses a **phased data strategy**:

- **Phase 1 & 2:** In-memory storage using Streamlit session state (temporary)
- **Phase 3:** Persistent storage using SQLite for tracking user progress across sessions

---

3.1 Data Model

The system uses a **relational SQLite database (real_estate_bot.db)** with three entities:

Entities

- **Users** → Stores user identity

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique user identifier
username	TEXT	UNIQUE, NOT NULL	User-chosen display name
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Account creation timestamp

- **Quiz Attempts** → Stores each quiz session

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique user identifier
username	TEXT	UNIQUE, NOT NULL	User-chosen display name
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Account creation timestamp

- **Question Responses** → Stores answers per question

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique response identifier
attempt_id	INTEGER	FOREIGN KEY → quiz_attempts(id)	Links response to an attempt
question_id	INTEGER	—	Identifier of the question from the question bank
topic	TEXT	—	Topic category of the question
difficulty	TEXT	—	Difficulty level: beginner, intermediate, or advanced
user_answer	TEXT	—	The option selected by the user (A, B, C, or D)
correct_answer	TEXT	—	The correct option for the question

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique response identifier
attempt_id	INTEGER	FOREIGN KEY → quiz_attempts(id)	Links response to an attempt
question_id	INTEGER	—	Identifier of the question from the question bank
topic	TEXT	—	Topic category of the question
difficulty	TEXT	—	Difficulty level: beginner, intermediate, or advanced
user_answer	TEXT	—	The option selected by the user (A, B, C, or D)
correct_answer	TEXT	—	The correct option for the question

Relationship

Users (1) → Quiz Attempts (many) → Question Responses (many)

This enables:

- User performance tracking
- Topic-wise analysis
- Long-term progress monitoring

3.2 Data Access Mechanism

Data access is handled via **database.py** using the **Repository Pattern**, ensuring separation of concerns.

Technology

- SQLite (sqlite3 – built-in Python library)
- Chosen for:
  - Lightweight usage
  - No server setup
  - Single-file database

Connection Handling

- Centralized get_connection()
- Uses sqlite3.Row for readability
- Uses with context → auto commit/rollback

Core Functions

- **init_db()** → Creates tables safely (idempotent)
- **get_or_create_user()** → Fetch or insert user
- **save_attempt()** → Stores quiz + responses (transaction-safe)
- **get_user_history()** → Fetch past attempts
- **get_weak_topics_historical()** → Identify weak areas (<60% accuracy)

3.3 Data Retention Policies

Storage & Security

- Database stored locally (.db file)
- Excluded via .gitignore
- API keys stored in .env

Retention

- Data stored **indefinitely** for learning analysis

AI Data

- AI explanations stored to:
  - Reduce API calls
  - Enable review

User Control

- "Delete My Data" option:
  - Deletes responses → attempts → user (cascade)

Data Collected

- Only username
- No sensitive data (email, IP, etc.)

3.4 Data Migration

Phase-wise Migration

- **Phase 1–2:** No database
- **Phase 3:** Tables created using init_db()

✓ Uses CREATE TABLE IF NOT EXISTS (safe & repeatable)

Future Changes

- Schema updates via:

ALTER TABLE question_responses ADD COLUMN time_taken_seconds INTEGER DEFAULT 0;

✓ Ensures backward compatibility

Migration Management

- Stored as versioned SQL scripts:

/migrations/migration_001.sql

✓ Enables:

- Version tracking
- Easy rollback
- Team collaboration

---

4. Interfaces

The RealEstate IQ Bot exposes **three types of interfaces**:

- User Interface
- Internal Module Interfaces
- External API Interface

4.1 User Interface

The application uses Streamlit to provide a web-based UI (localhost:8501) with four main screens:

1. Welcome Screen

- Displays app overview (topics, questions, AI model)
- "Start Quiz" button to begin

---

2. Filter Screen

- Select:
  - Topics (7 options)
  - Difficulty levels (3 levels)
- Shows live question count
- Prevents empty quiz (disabled start button if no match)

---

3. Quiz Screen

- Displays:
  - One question at a time
  - Progress bar
  - Topic & difficulty badges
- Input via radio buttons
- No page reload (dynamic update)

---

4. Result Panel

- Shown after answer submission

- Displays:
  - Correct/Incorrect status
  - Correct answer
  - AI explanation
  - Key concept & tip
- "Next Question" button

---

5. Summary Dashboard

- Displays:
  - Score & accuracy
  - Grade (A–D)
  - Topic-wise & difficulty-wise performance
  - Weak topics (<60%)
- Includes:
  - Answer review section
  - "Retake Quiz" option

4.2 Internal Module Interfaces

Modules interact through **well-defined function calls**.

Key Interface

- **evaluate_answer() (evaluator.py)**
  - Input: question + user answer
  - Output:
    - is_correct
    - score
    - key_concept
    - explanation
    - pro_tip

✓ Ensures consistent response format

✓ Includes fallback handling (same structure even if API fails)

4.3 External API Interface

The system integrates with **Groq API** for AI evaluation.

Usage

- Evaluates answers
- Generates explanations
- Provides learning tips

5. State and Session Management

The RealEstate IQ Bot manages application state using Streamlit session_state, which supports interactive, single-user sessions.

5.1 State Management Approach

All runtime data is stored in session_state, ensuring consistency without global variables.

Tracked Data

- Current question index
- Total score
- Answer history
- Topic-wise performance
- Weak topics (<60% accuracy)

✓ Maintains continuity across Quiz, Result, and Dashboard screens

---

5.2 Session Lifecycle

- **Start:** User begins quiz → state initialized
- **During Quiz:** State updated after each answer
- **End:** Summary generated

Session Ends When

- Browser refresh
- Manual reset (Retake Quiz)

5.3 Session Manager Role

The session_manager.py module:

- Updates score and progress
- Tracks topic-wise accuracy
- Identifies weak areas

✓ Centralized control ensures consistency

5.4 Future Enhancement

- Shift from in-memory → SQLite persistence
- Enables:
  - Cross-session tracking
  - Personalized learning
  - Historical analysis

---

6. Caching

Caching is used to improve performance and reduce repeated API calls.

6.1 Need for Caching

The system uses external LLM APIs (e.g., Groq API), which:

- Add latency
- Have request limits

✓ Caching improves:

- Response time
- Efficiency
- User experience

6.2 Current Caching Strategy

- AI responses stored in session history
- Reused within the same session

✓ Avoids duplicate API calls

6.3 Future Caching Enhancements

1. Persistent Caching

- Store responses in SQLite
- Reuse across sessions

---

2. Streamlit Caching

- Use:
  - st.cache_data
  - st.cache_resource
- For static data (questions/configs)

---

3. Smart Caching

- Cache based on:
  - Question ID
  - User answer

✓ Same input → same output reuse

6.4 Benefits of Caching

- Reduced latency
- Lower API usage
-
- Better scalability
- Improved user experience

---

7. Non-Functional Requirements

Non-functional requirements define system quality attributes such as **security, performance, reliability, and scalability**, ensuring efficient and safe operation of the RealEstate IQ Bot.

7.1 Security Aspects

The system implements multiple security measures:

API Key Security

- API key stored in .env file
- .env excluded via .gitignore
- Prevents exposure in version control

Data Privacy

- Only username is collected
- No sensitive data (email, IP, etc.) stored
- Data remains local to the system

Secure Data Storage

- SQLite database stored locally
- Database file excluded from version control
- Prevents unauthorized access

Controlled External Communication

- Only external service: Groq API
- Sends only required data (question & answer)
- No unnecessary data sharing

Error Handling & Safety

- Exception handling prevents crashes
- Ensures stable execution

7.2 Performance Aspects

The system is optimized for speed and efficiency:

Low Latency Responses

- Groq API provides fast inference
- Enables near real-time feedback

Efficient Resource Usage

- In-memory session management reduces overhead
- Lightweight Streamlit UI ensures smooth performance

Optimized API Usage

- Stores responses in session to avoid repeated calls
- Future caching will further reduce API usage

Scalability

- Modular architecture supports:
    - Multiple users
    - Database integration
    - Cloud deployment

Reliability

- Retry logic handles API limits/failures
- Ensures consistent system performance

---

8. References

- Groq API Docs
- Streamlit Documentation
- Python Docs
- Groq
- Groq API Documentation
- Available at: https://console.groq.com/docs
- LLaMA 3
- Meta AI, LLaMA 3 Model Documentation
- Available at: https://ai.meta.com
- Prompt Engineering Guide
- Available at: https://platform.openai.com/docs/guides/prompt-engineering
- Real Estate Principles & Practices — Industry reference material

---